

TP2 – Comunicación de Tareas de FreeRTOS

Objetivos

Usar **Comunicación de Tareas** e **Interrupciones** de **FreeRTOS**

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Editar/compilar/depurar programas en **C** con **FreeRTOS**

Configurar el **IDE** para depurar en modo **semihosting**

Adoptar metodología orientada a la portabilidad/reúso de **código**

Identificar **Patrones de Diseño de Software**

Usar **Git** y **GitHub**

Generar/actualizar **repositorio** por línea de **comando**

Generar/actualizar **archivos** del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/explicar **código fuente**

Comunicación de Tareas e Interrupciones

Cómo crear una **cola**.

Cómo gestiona una **cola** los datos que contiene.

Cómo enviar datos a una **cola**.

Cómo recibir datos de una **cola**.

Qué significa bloquearse en una **cola**.

Cómo bloquearse en varias **colas**.

Cómo sobrescribir datos en una **cola**.

Cómo vaciar una **cola**.

El efecto de las prioridades de las **Tareas** al escribir y leer en una cola.

Cómo crear y usar **semáforos binarios** y **semáforos contadores**.

Las diferencias entre **semáforos binarios** y **semáforos contadores**.

Qué funciones de la API de FreeRTOS se pueden usar dentro de una rutina de servicio de **interrupción**.

Métodos para delegar el procesamiento de **interrupciones** a una **Tarea**.

Cómo usar una **cola** para transferir datos dentro y fuera de una rutina de servicio de **interrupción**.

El modelo de anidamiento de **interrupciones** disponible en algunas portaciones de **FreeRTOS**.

Actividades

1er encuentro

TP2-01 – 4to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

TP2-02 – 5to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

2do encuentro

TP2-02 – 6to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

TP2-03 – 7mo Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Resolución en grupos de **3** alumnos, requiere **presentación** por parte de un **responsable** del grupo

TP2 – Actividad 01 – 4to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

Paso 01: Crear un nuevo **repositorio** en **GitHub** para almacenar su **TP2**, pasos:

GitHubRepositories => **New** =>

Repository name: **soe-tp2_****Año-Cuatrimstre****_Curso-Grupo**

Description: **# FIUBA - Electrónica - Sistemas Operativos Embebidos - Trabajo Práctico N°: 2 – Comunicación de Tareas de FreeRTOS**

Initialize this repository with:

Tildar **Add a README file**

=> **Create repository**

Año-Cuatrimstre es: **2026-1erC** o **2026-2doC** ... etc. (el correspondiente al Año y Cuatrimstre de cursada)
Curso-Grupo es: **1-01** o **1-02** ... etc. (el correspondiente al Curso y Grupo de cursada)

Paso 02: Invitar al **repositorio**, como **colaboradores** a los **docentes auxiliares** de su curso, responsables de la aprobación de su **TP2**, pasos:

GitHub: **Repositories** => **Settings** => **Collaborators** => Tildar: **Add people** =>

en **Find people** => tipear: **Docente_Auxiliares-FIUBA**

Tildar: **Add Docente_Auxiliares-FIUBA**

Una vez que los **docentes auxiliares** de su curso, **hayan aceptado** la **invitación** a ser **colaboradores**, podrá agregarlos como **Revisores**

PasPaso 03: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **editar** el archivo **README.md** y **modificar** su contenido, pasos:

GitHub: **READM.me** => **Edit this file** => **borrar** su contenido => **Copiar** y **Pegar** lo siguiente:

```
# FIUBA - Electrónica - Sistemas Operativos Embebidos
## Trabajo Práctico N°: 2 - Comunicación de Tareas de FreeRTOS
### Año-Cuatrimestre - Curso-Grupo

### Responsable de la entrega:
| Padrón | Apellidos, Nombres | Fecha | Deadline |
| :----- | :----- | :----- | :----- |
| XXXXXX | YYYY, ZZZ | | Semana 06 |
```

Actualizar el archivo **README.md** con los datos del responsable de la entrega (un alumno diferente para cada TP):

Año-Cuatrimestre, **Curso-Grupo**, **Padrón**, **Apellidos** y **Nombres**

Paso 04: Generar un nuevo **proyecto STM32**: descargar el archivo [soe-tp2_01-application.zip](#) e importar el **proyecto STM32** que contiene: [soe-tp2_01-application](#).

Compilar el **proyecto STM32** (**STM32CubeIDE**) importado: [soe-tp2_01-application](#).

Paso 05: Asentar los detalles de las entregas en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), crear el archivo [soe-tp2_01-application.md](#).

Paso 06: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Logear, en **Google** con tu cuenta de correo electrónico institucional, del dominio [@fi.uba.ar](#).

Ingresar a **Gemini** => <https://gemini.google.com/app>, => **Nueva Conversación** => elegir modelo **Pro** => en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [startup_stm32f103rbtx.s](#), [main.c](#), [stm32f1xx_it.c](#), [FreeRTOSConfig.h](#) y [freertos.c](#).

Indicar la evolución de las variables **SysTick** y **SystemCoreClock** al ejecutar dicho código fuente desde su inicio (**Reset_Handler**: de [startup_stm32f103rbtx.s](#)) hasta el loop principal de la aplicación (**while (1)** de [main.c](#)).

Indicar el comportamiento del programa al ejecutar dicho código fuente desde su inicio (**Reset_Handler**: de [startup_stm32f103rbtx.s](#)) hasta antes de llegar al loop principal de la aplicación (**while (1)** de [main.c](#)).

Indicar cómo y para qué **SysTick** y **Timer 1 (TIM2)** interactúan con **FreeRTOS**.

Indicar cómo y para qué el **Timer 4 (TIM4)** interactúa con la **HAL** del **proyecto STM32**.

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...\soe_workspace\soe-tp2_01-application\Core\Src\main.c
...\soe_workspace\soe-tp2_01-application\Core\Src\stm32f1xx_it.c
...\soe_workspace\soe-tp2_01-application\Core\Src\freertos.c
...\soe_workspace\soe-tp2_01-application\Core\Inc\FreeRTOSConfig.h
...\soe_workspace\soe-tp2_01-application\Core\Startup\startup_stm32f103rbtx.s
```

Leer y almacenar la respuesta en: [...\soe_workspace\soe-tp2_01-application\soe-tp2_01-application.md](#).

Paso 07: Depurar el nuevo **proyecto STM32**:

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso TP2 – Actividad 01 – Paso 06**.

Paso 06: Consultar a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

Ingresar a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

Copiar y Pegar la siguiente línea de **texto**:

Analizar y explicar (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [task_btn.c](#), [task_led.c](#), [task_led_interface.c](#), y [freertos.c](#).

Agregar los siguientes archivos (click en **+** Agregar archivos => **Subir archivos**):

```
...\soe_workspace\soe-tp2_01-application\Core\Src\app.c  
...\soe_workspace\soe-tp2_01-application\app\src\task_btn.c  
...\soe_workspace\soe-tp2_01-application\app\src\task_led.c  
...\soe_workspace\soe-tp2_01-application\app\src\task_led_interface.c  
...\soe_workspace\soe-tp2_01-application\app\src\freertos.c
```

Leer y almacenar la respuesta en: [...\soe_workspace\soe-tp2_01-application\soe-tp2_01-application.md](#).

Paso 09: Depurar el nuevo **proyecto STM32**.

Confirmar mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de **Gemini Pro** del **Paso TP2 – Actividad 01 – Paso 08**.

Paso 10: Almacenar el **proyecto STM32**: [soe-tp2_01-application](#) y el/los archivo/s de la actividad: [soe-tp2_01-application.md](#), en el **repositorio** creado en **GitHub**, en el **TP2 – Actividad 01 – Paso 01**.

TP2 – Actividad 02 – 5to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar Comunicación de Tareas de FreeRTOS

Crear/Gestionar/Comunicar Tareas (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias) con **Cola** (bloqueantes o no bloqueantes)

Usar STM32CubeIDE

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar Git y GitHub

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: **Generar** un nuevo **proyecto STM32**: **copiar** el archivo **soe-tp2_01-application** y **pegar/renombrar** como **soe-tp2_02-application**.

Renombrar el archivo **soe-tp2_01-application.ioc** como **soe-tp2_02-application.ioc**.

Borrar los archivos de compilación/programación/depuración (**soe-tp2_01-application.launch** y **soe-tp2_01-application.cfg**).

Compilar/Depurar el **proyecto STM32** (**STM32CubeIDE**) renombrado (**soe-tp2_02-application**).

Generar los archivos de depuración (**soe-tp2_02-application.launch** y **soe-tp2_02-application.cfg**).

Paso 02: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **soe-tp2_02-application.md**, **editar** y **modificar** su contenido respondiendo las siguientes preguntas (en base a su experiencia depurando el **proyecto STM32**):

- ¿Cómo eliminar una **Cola**?
- ¿Cómo crear una **Cola**?
- ¿Cómo gestiona una **Cola** los datos que contiene?
- ¿Cómo enviar datos a una **Cola**?
- ¿Cómo recibir datos de una **Cola**?
- ¿Qué significa bloquearse en una **Cola**?
- ¿Cómo bloquearse en varias **Cola**?
- ¿Cómo sobrescribir datos en una **Cola**?
- ¿Cómo vaciar una **Cola**?
- ¿Cuál es el efecto de las prioridades de las **Tareas** al escribir y leer en una **Cola**?

Paso 03: **Modificar** el mecanismo de comunicación entre las **task_btn** y **task_led**, recurrir a una **Cola** (transmisión y recepción, bloqueante o no bloqueante, lo adecuado), **compilar/depurar/observar** comportamiento, **asentar** lo observado en el archivo **soe-tp2_02-application.md**.

Paso 04: **Almacenar** el **proyecto STM32**: **soe-tp2_02-application** y el/los archivo/s de la actividad: **soe-tp2_02-application.md**, en el **repositorio** creado en **GitHub**, en el **TP2 – Actividad 01 – Paso 01**.

TP2 – Actividad 03 – 6to Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar **Comunicación de Tareas de FreeRTOS**

Crear/Gestionar/Comunicar Tareas (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias) con **Semáforo binario** (bloqueantes o no bloqueantes)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: **Generar** un nuevo **proyecto STM32**: **copiar** el archivo **soe-tp2_01-application** y **pegar/renombrar** como **soe-tp2_03-application**.

Renombrar el archivo **soe-tp2_01-application.ioc** como **soe-tp2_03-application.ioc**.

Borrar los archivos de compilación/programación/depuración (**soe-tp2_01-application.launch** y **soe-tp2_01-application.cfg**).

Compilar/Depurar el **proyecto STM32** (**STM32CubeIDE**) renombrado (**soe-tp2_03-application**).

Generar los archivos de depuración (**soe-tp2_03-application.launch** y **soe-tp2_03-application.cfg**).

Paso 02: **Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **soe-tp2_03-application.md**, **editar** y **modificar** su contenido respondiendo las siguientes preguntas (en base a su experiencia depurando el **proyecto STM32**):

¿Cómo crear y usar **semáforos binarios** y **semáforos contadores**?

¿Cuáles son las diferencias entre **semáforos binarios** y **semáforos contadores**?

Paso 03: **Modificar** el mecanismo de comunicación entre las **task_btn** y **task_led**, recurrir a una **Semáforo binario** (give y take, bloqueante o no bloqueante, lo adecuado) **compilar/depurar/observar** comportamiento, **asentar** lo observado en el archivo **soe-tp2_03-application.md**.

Paso 04: **Almacenar** el **proyecto STM32**: **soe-tp2_03-application** y el/los archivo/s de la actividad: **soe-tp2_03-application.md**, en el **repositorio** creado en **GitHub**, en el **TP2 – Actividad 01 – Paso 01**.

TP2 – Actividad 04 – 7mo Proyecto p/placa NUCLEO-F103RB con FreeRTOS

Objetivos

Usar Comunicación de Tareas de FreeRTOS

Crear/Gestionar/Comunicar Tareas (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias) e **Interrupciones** con **Cola** o **Semáforo binario** (bloqueantes o no bloqueantes)

Usar STM32CubeIDE

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar Git y GitHub

Generar/Actualizar repositorio por línea de **comando**

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Paso 01: Generar un nuevo **proyecto STM32**: **copiar** el archivo **soe-tp2_01-application** y **pegar/renombrar** como **soe-tp2_04-application**.

Renombrar el archivo **soe-tp2_01-application.ioc** como **soe-tp2_04-application.ioc**.

Borrar los archivos de compilación/programación/depuración (**soe-tp2_01-application.launch** y **soe-tp2_01-application.cfg**).

Compilar/Depurar el **proyecto STM32** (**STM32CubeIDE**) renombrado (**soe-tp2_04-application**).

Generar los archivos de depuración (**soe-tp2_04-application.launch** y **soe-tp2_04-application.cfg**).

Paso 02: Asentar los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **soe-tp2_04-application.md**, **editar** y **modificar** su contenido respondiendo las siguientes preguntas (en base a su experiencia depurando el **proyecto STM32**):

¿Qué funciones de la API de FreeRTOS se pueden usar dentro de una rutina de servicio de **interrupción**?

¿Métodos para delegar el procesamiento de **interrupciones** a una **Tarea**?

¿Cómo usar una **cola** para transferir datos dentro y fuera de una rutina de servicio de **interrupción**?

¿Cuál es el modelo de anidamiento de **interrupciones** disponible en algunas portaciones de **FreeRTOS**?

Paso 03: Modificar **task_btn** para atender el botón por interrupción, como mecanismo de comunicación entre la rutina de servicio de **interrupción** y **task_btn**, recurrir a una **Semáforo binario** (give y take, bloqueante o no bloqueante, lo adecuado) como mecanismo de comunicación, **compilar/depurar/observar** comportamiento y **asentar** lo observado en el archivo **soe-tp2_04-application.md**.

Paso 04: Almacenar el **proyecto STM32**: **soe-tp2_04-application** y el/los archivo/s de la actividad: **soe-tp2_04-application.md**, en el **repositorio** creado en **GitHub**, en el **TP2 – Actividad 01 – Paso 01**.

GitHub: Presionar: **Commit changes ...** (Guardar los cambios)

Tildar: **Create a new branch** for this commit and start a pull request

Presionar: **Propose changes**

Presionar: **Create pull request**, tipear el nombre del **branch** y **agregar** a los **docentes auxiliares** de su curso, como **Reviewers** (para invitarlos a **revisar** sus **entregas** mediante un **pull request**)

Ver **cómo hacer el pull request** en este video: [Trabajo colaborativo con Github](#)

En la descripción del video se detalla el contenido (ver en [14:23](#) la sección "CÓMO SOLICITAR REVISIONES")
IMPORTANTE: Se recomienda que primero vean "[19:25](#) Solución a un problema muy habitual" y luego "[19:02](#)
Create pull request y Review requested"